USER_GUIDE

Challenges Module - User Guide

Creating a Challenge

1. Implement the Challenge Interface

```go
package mychallenges

import (
    "context"
    "digital.vasic.challenges/pkg/challenge"
)

type APIHealthChallenge struct {
    challenge.BaseChallenge
    apiURL string
}

func NewAPIHealthChallenge() *APIHealthChallenge {
    return &APIHealthChallenge{
        BaseChallenge: *challenge.NewBaseChallenge(
            "api_health",
            "API Health Check",
            "Verifies the API is responding correctly",
            "core",
            nil, // no dependencies
        ),
    }
}

func (c *APIHealthChallenge) Execute(
    ctx context.Context,
) (*challenge.Result, error) {
    result := c.CreateResult()
    result.Status = challenge.StatusRunning
```

```
    // Your test logic here
    resp, err := http.Get(c.apiURL + "/health")
    if err != nil {
        result.Status = challenge.StatusFailed
        result.Error = err.Error()
        return result, nil
    }
    defer resp.Body.Close()

    // Set assertions
    result.Assertions = c.EvaluateAssertions(
        []challenge.AssertionDef{
            {Type: "min_count", Target: "status_code",
                Value: 200, Message: "Should return 200"},
        },
        map[string]any{"status_code": resp.StatusCode},
    )

    // Set metrics
    result.Metrics["response_code"] = challenge.MetricValue{
        Name: "response_code", Value: float64(resp.StatusCode),
    }

    result.Status = challenge.StatusPassed
    return result, nil
}
```

2. Register the Challenge

```
import "digital.vasic.challenges/pkg/registry"

reg := registry.NewRegistry()
reg.Register(NewAPIHealthChallenge())
```

3. Run Challenges

```
import "digital.vasic.challenges/pkg/runner"

r := runner.NewRunner(
    runner.WithRegistry(reg),
    runner.WithTimeout(5 * time.Minute),
    runner.WithResultsDir("./results"),
)

// Run all in dependency order
results, err := r.RunAll(ctx, &challenge.Config{
    Verbose: true,
})

// Run specific challenges
```

```
results, err = r.RunSequence(ctx,
    []challenge.ID{"api_health", "db_health"},
    config,
)

// Run in parallel
results, err = r.RunParallel(ctx,
    []challenge.ID{"api_health", "db_health"},
    config, 4, // max concurrency
)
```

4. Generate Reports

```
import "digital.vasic.challenges/pkg/report"

// Markdown report
mdReporter := report.NewMarkdownReporter("./reports")
data, _ := mdReporter.GenerateMasterSummary(results)

// JSON report
jsonReporter := report.NewJSONReporter("./reports", true)
data, _ = jsonReporter.GenerateReport(results[0])

// HTML report
htmlReporter := report.NewHTMLReporter("./reports")
data, _ = htmlReporter.GenerateMasterSummary(results)
```

Using Shell Challenges

Wrap existing bash scripts as challenges:

```
shell := challenge.NewShellChallenge(
    "verify_providers",
    "Provider Verification",
    "Runs provider verification script",
    "validation",
    nil,
    "/path/to/verify.sh",
    []string{"--verbose"},
    "/working/dir",
)
reg.Register(shell)
```

Using the Assertion Engine

```
import "digital.vasic.challenges/pkg/assertion"

engine := assertion.NewEngine()
```

```
// Evaluate single assertion
result := engine.Evaluate(
    assertion.Definition{
        Type:   "contains",
        Target: "response",
        Value:  "success",
    },
    "Operation completed successfully",
)

// Register custom evaluator
engine.Register("custom_check", func(
    def assertion.Definition, value any,
) (bool, string) {
    // Your logic
    return true, "custom check passed"
})
```

Using the Plugin System

```
import "digital.vasic.challenges/pkg/plugin"

type MyPlugin struct{}

func (p *MyPlugin) Name() string    { return "my-plugin" }
func (p *MyPlugin) Version() string { return "1.0.0" }

func (p *MyPlugin) RegisterChallenges(
    reg registry.Registry,
) error {
    return reg.Register(NewMyChallenge())
}

func (p *MyPlugin) RegisterAssertions(
    engine assertion.Engine,
) error {
    return engine.Register("my_assert", myEvaluator)
}

// Load plugins
pluginReg := plugin.NewPluginRegistry()
pluginReg.Register(&MyPlugin{})
pluginReg.LoadAll(challengeRegistry, assertionEngine)
```

Challenge Banks

Load challenge definitions from JSON files:

```
{
    "version": "1.0",
    "challenges": [
        {
            "id": "api_health",
            "name": "API Health Check",
            "category": "core",
            "dependencies": [],
            "assertions": [
                {"type": "not_empty", "target": "response"}
            ]
        }
    ]
}
```

```
import "digital.vasic.challenges/pkg/bank"

b := bank.NewBank()
b.Load("challenges_bank.json")
defs := b.ListByCategory("core")
```

Live Monitoring

```
import "digital.vasic.challenges/pkg/monitor"

collector := monitor.NewEventCollector()
ws := monitor.NewWebSocketServer(collector, ":8090")
go ws.Start()

// Events are automatically collected during runner execution
```